

Runi - Team30

Runi - Team30 (*student proposal*)

Assessment: ✓ Fit · **Category:** Games (*gamified-app lens*) · **Assessed:** 2026-09-22 · **Status:** awaiting instructor approval

Proposal (as submitted): *Runi* is a **gamified running platform** that turns real-world running into an interactive experience with challenges, progression, achievements and social interaction to build consistent habits. Named load-bearing C++ (the **Runi Activity Engine**): a subsystem that processes incoming **GPS** samples, filters noisy readings, calculates distance and pace, tracks route progress, **interpolates ghost positions** and generates real-time **race events** (distance-milestone reached, personal-best split beaten, ghost overtaken, challenge objective completed). Named technical features: **GPS Route Tracking & Ghost Racing** (race a "ghost" of your previous/personal-best run or a friend's, with live ahead/behind stats on completed routes); **Virtual Pet Progression & Reward** (a persistent pet whose XP/accessories/cosmetics are driven by streaks, distance milestones, specific routes and challenges); **Social Network & Competitive Leaderboards** (friends, shared activities, challenges, leaderboards by route-time/distance/achievements); and an **Achievement & Challenge System** (configurable objectives → XP/achievements/cosmetics). Tools: a **content-authoring tool** (configure challenges, achievements, progression values, cosmetics and rewards as data, distributed via the backend — a data-driven workflow); and a **testing tool** that **simulates running activity** (GPS movement, distance, pace, completed runs) so developers can test/debug/balance ghost racing, challenges, rewards and pet progression without physically running, with adjustable speed/distance/progression. Design features: a Running Dashboard, an Adaptive Goal System, Friends & Community, a Challenge Hub, Personal Racing (live ahead/behind), a Virtual Pet & Profile (raise/customise/equip cosmetics), and Pet Activity States (happy/energetic/sleepy/exhausted; hatch + evolve on milestones). Team: **6 CS + 1 Design + 0 Art**.

Note (category): like other **gamified apps** (there is no fitness/health category), *Runi* is filed under **Games** and assessed through the gamified-app lens (session/

progression/retention systems) — the same way the module treats step-and-pet and habit-loop projects. (A Social Media reading was considered for the friends/leaderboard layer, but the core is the gamified running/pet loop.)

Note (module completeness): Runi needs a **backend + external DB** (persistent pet/XP/cosmetics, runs, leaderboards, social graph, content packs) — use Firebase and/or the team's Linux server (set the seam up + basic-test it by M2).

Platform: GPS + on-the-go use → a **mobile** build (Native Android, Pixel 9a).

Monetization: none specified and the economy is **earn-by-running** (XP/cosmetics), which is clean; if real-money purchases are added, keep them **cosmetic / non-pay-to-win** per the module rule.

References:

- [Strava — Wikipedia](#) (the benchmark GPS running/cycling app with route + pace tracking, segment **leaderboards** and a **social** feed where users follow each other and compete on segment times — the GPS-tracking + social + leaderboard + segment-racing precedent)
- [Tamagotchi — Wikipedia](#) (the archetypal **virtual pet** you raise and customise, whose **states and branching evolution** depend on how it's cared for — the persistent-pet + activity-driven-states + hatch/evolve precedent)
- [TrackMania — Wikipedia](#) (the racing benchmark for **ghost racing**: race against a recorded ghost of your best time or others' times, submitted to online leaderboards — the ghost-of-a-previous-run + live-comparison precedent)

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

Core Tech Requirement	Runi
External Database	✓
Authentication »	●
Analytics »	●
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	✓
2D Graphical Output	✓
3D (strong students only)	✕
Networking System	●

How each requirement applies:

Platform fit: **Main:** Native mobile (Native Android — C++ Activity Engine via JNI, OpenGL ES for the map/ghost/pet rendering in a GLSurfaceView, Kotlin app shell; device: Pixel 9a) — GPS, background-safe run recording and on-device state are native concerns. The Native PC build is the dev/tools vehicle (the content-authoring tool + the GPS-simulation testing tool). **Extended:** Firebase + a Linux server the team sets up by M2 (full use T2) for pet/XP/cosmetic persistence, runs, leaderboards, the social graph and content-pack distribution.

Front end: A 2D app — the Running Dashboard, the route map with the live ghost + ahead/behind feedback, the Challenge Hub, leaderboards/friends feed, and the virtual pet (states, cosmetics, evolution). **Languages:** Native — Kotlin Native Android UI + the C++ engine via JNI (Android Studio); OpenGL ES for the realtime map/ghost/pet in a GLSurfaceView; dashboard/social/profile screens can be pure Native UI. Web/desktop — SDL2/3 + OpenGL ES for the PC dev/tools build.

Back end: The C++ Runi Activity Engine — GPS ingest + noise filtering, distance/pace, route-progress tracking, ghost-position interpolation and real-time race-event generation — plus the pet-progression, achievement/challenge and leaderboard systems; a backend (Firebase/Linux server) for persistence, social and content packs. **Languages:** C++ (engine); Firebase/server on the Extended tier.

Backend scope: Local now, server-backed (T2) — the persistence/social/content seam set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* the C++ Activity Engine is the heart — per GPS sample it filters noise, updates distance/pace + route progress, interpolates the ghost, and fires race events (milestone, PB split, ghost overtaken, challenge complete).
- *Data-driven:* challenges, achievements, progression curves, cosmetics, rewards and pet-evolution rules are all **content packs** authored in the tool and distributed via the backend — new content ships without code changes (a named requirement).
- *2D:* the dashboard, the route map + ghost, the challenge/leaderboard screens and the virtual pet (states, cosmetics) are custom-rendered. *3D:* not needed.
- *Front-end + Back-end:* a client/server split — device app + backend for persistence, leaderboards, social and content — the run-upload + content-distribution flows are **Application Communication** ✓.
- *Custom tools:* two — the **content-authoring tool** (challenges/achievements/progression/cosmetics as data) and the **GPS-simulation testing tool** (synthetic runs to test ghost racing/challenges/rewards/pet without running).
- *Networking:* single-player with async social — runs upload and leaderboards/social sync after the fact, so a real-time Networking System is optional (●). *External DB:* central (pet/runs/leaderboards/social/content).

Auth » / Analytics »: accounts + activity analytics are natural but module-deferred to T2 (●).

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Games pool in [milestone-goals-pool-per-category.pdf](#) (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic** — the **Runi Activity Engine** (GPS filtering, distance/pace, ghost interpolation, race events).

Tech & dependencies

- **Tech: Web build:** SDL2/3 window + a C++ loop processing GPS samples (noise filtering), computing distance/pace + route progress, interpolating ghost positions and generating race events; C++17/20, CMake, Git. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); GPS + OpenGL ES map in a GLSurfaceView; some screens pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; the ghost replay (M1G15) and rendering (M1G4) build on it.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**

Tech & dependencies

- **Tech:** An entity/data model with challenges, achievements, progression curves, cosmetics, rewards and pet-evolution rules in hot-reloadable, versioned JSON content packs — a new challenge or cosmetic is a data change.
- **Prerequisites:** G1 (loop & entity hooks).
- **Feeds into:** M2G2 (the content-authoring tool), M2G13 (pet/challenge systems).

- **G3 ★ Debugging, Profiling, Stress Test & CI** — incl. the **GPS-simulation testing tool**.

Tech & dependencies

- **Tech:** A testing tool feeding synthetic GPS traces (adjustable speed/distance/progression) through the real Activity Engine, ImGui overlays (route progress, ghost delta, race events), Tracy profiling of a long run,

Catch2 tests (GPS filtering, distance/pace, event firing), on a GitHub Actions matrix.

- **Prerequisites:** G1 (engine to drive).
- **Feeds into:** M2G3 (CI hardening); the testing tool at showcase.

- **G4 2D Graphics** — the dashboard, the route map + live ghost, and the virtual pet.

Tech & dependencies

- **Tech: Web build:** A renderer for the route map + the ghost marker with ahead/behind feedback, the dashboard, and the virtual pet (states/cosmetics), with a camera; SDL2/3/OpenGL ES, GLSL, FreeType. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES map/pet in a GLSurfaceView; some screens pure Native UI.
 - **Prerequisites:** G1 (loop & window).
 - **Feeds into:** Every visible frame; the pet/cosmetic display.
- **G12 Save / Persistence & Firebase Integration** — persistent pet/XP/cosmetics, runs, leaderboards, social.

Tech & dependencies

- **Tech:** Local persistence of runs + pet/XP/cosmetics, synced to Firebase (Firestore) for the social graph, shared activities, leaderboards and content-pack distribution; REST from C++ (JS SDK on web).
 - **Prerequisites:** G2 (data model to sync).
 - **Feeds into:** M2G8 (accounts/leaderboards/analytics); reliable progression + social.
- **G15 Deterministic Input & Replay Foundation** — the **ghost** recording/replay.

Tech & dependencies

- **Tech:** Record a run as a timestamped position/pace track and replay it deterministically as a ghost, with live ahead/behind computation against the current run; the basis for personal-best and friends' ghosts.
- **Prerequisites:** M1G1 (the Activity Engine), M1G3 (debug hooks).
- **Feeds into:** M2G12 (Replay System — multi-ghost, friends' ghosts); the Personal Racing feature.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation** — battery-aware on-device perf + the server seam.

Tech & dependencies

- **Tech: Web build:** Battery-aware GPS/render cadence, memory pools, Tracy profiling of a long run, ASan on MSVC, and on-device validation (Pixel 9a); the Firebase/Linux-server seam (persistence/social/content) set up (account + software + basic connectivity test) by M2. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES map/pet in a GLSurfaceView; some screens pure Native UI.
- **Prerequisites:** All M1 goals (hardens the M1 codebase).
- **Feeds into:** The mobile deliverable + a battery-viable long run.

- **G2 ★ Content Editor — Core (Content-Authoring Tool)** — challenges, achievements, progression, cosmetics, rewards as data.

Tech & dependencies

- **Tech:** Dear ImGui (or a small external editor): author challenges/achievements/progression values/cosmetics/rewards, round-trip JSON content packs, live preview and undo/redo, distributed via the backend.
- **Prerequisites:** G2 (data model).
- **Feeds into:** All live content; the custom-tools showcase.

- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, reproducible Windows + Android builds, tests (GPS filtering, distance/pace, ghost delta, event firing, content-pack loading — driven by the GPS-simulation tool) and packaging via GitHub Actions. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES map/pet in a GLSurfaceView; some screens pure Native UI.
- **Prerequisites:** G3 (CI & test foundation).
- **Feeds into:** Release/showcase builds of every goal.

- **G8 Firebase — Accounts, Leaderboards & Analytics** — the social network + competitive leaderboards.

Tech & dependencies

- **Tech:** Firebase Auth + Firestore: accounts, the friend/social graph, shared activities + reactions, group challenges, and the competitive leaderboards (route time / distance / achievements), server-validated so scores can't be forged.
- **Prerequisites:** M1G12 (persistence/sync).
- **Feeds into:** Friends & Community + Challenge Hub; T2 live-ops.
- **G12 Replay System — Record & Playback (Ghost Racing)** — the ghost racing matured.

Tech & dependencies

- **Tech:** Multi-ghost replay (previous run, personal best, friends' ghosts) with robust live ahead/behind, race-event triggers (ghost overtaken, PB split) and shareable ghost tracks.
- **Prerequisites:** M1G15 (replay foundation).
- **Feeds into:** The Personal Racing showcase; competitive route leaderboards.
- **G13 Gameplay Systems II** — the **virtual pet + XP/cosmetics + achievements/challenges** systems.

Tech & dependencies

- **Tech:** C++ systems over the entity/data model: the pet progression (XP, states happy/energetic/sleepy/exhausted, hatch/evolve on milestones), the cosmetic inventory + equip, and the achievement/challenge objective engine + reward issue; balancing-data-driven via content packs.
- **Prerequisites:** M1G2 (data), M1G1 (activity events).
- **Feeds into:** The pet/progression showcase; T2 content expansion.

Track Goals & Team Composition

Recommended composition: 6 CS + 1 Design + **1 Art** (8) — the proposed **6 CS + 1 Design + 0 Art** is the module's tight case for an **art-heavy** concept: a virtual pet with **states, cosmetics/accessories and hatch/evolve stages**, plus dashboard/map/UI, is a real art surface, so with no artist the single designer must also carry the **D11 conditional** each milestone, and a **solo designer + 0 artists** is exactly what the design pool flags ("must ask the instructor which conditional bends", see [design-goals-pool.pdf](#)). **Strongly recommended: add 1 Artist** (best for the pet/cosmetic/evolution volume), or at least a **2nd designer** so one owns the run/challenge/retention systems and the other absorbs the D11 art surface. If the team stays 0-artist, scope the art hard: **one pet with a few evolution stages and a small, reusable, recolourable cosmetic set** authored/sourced under D11, and expect the instructor sign-off on the bent conditional.

Scope note (advisory): a GPS Activity Engine (noise filtering + ghost interpolation + race events) + a replay/ghost system + persistence/social/leaderboards + a data-driven content pipeline + two tools is a **large, server-backed CS scope** — GPS→distance/pace, one recordable ghost with ahead/behind, basic pet progression and one editor is the M1/M2 floor; friends' ghosts, the full social/leaderboard layer, pet evolution and all cosmetics/challenges are the stretch. The **content-pack pipeline + the GPS-simulation testing tool are the smartest levers** — they let a small team add content as data and test without running.

Track goals — suggested Design / UI-UX / TEAM picks for this project (no Art track as proposed — 0 artists, so the pet/cosmetic Art surface is carried by the Design-owned D11 conditional; adding an artist is recommended, see above); deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the gamified-running USP, ghost racing, the virtual-pet progression, the challenge/achievement economy and the social/leaderboard layer; primary benchmark e.g. Strava / a virtual-pet fitness app, plus a Singapore benchmark — Healthy 365 fits the local-awareness criterion)
- M1D9 Engagement & Retention Loop Design (the streaks, adaptive goals, challenges and pet-progression retention — with the ethics line: engagement serves the user's fitness goal, never dark-pattern or pay-to-win)

- M1D11 Art Proof of Concept *(conditional — mandatory for 0-artist teams; Design-owned)* — the pet visual direction (states + evolution stages), the cosmetic set, and the dashboard/map UI style; 2 mockups.

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D11 Art Implementation Quality *(conditional — mandatory for 0-artist teams; Design-owned)* — the pet (states/evolution), cosmetics and UI implemented in-engine, no placeholders.

- **UI/UX M1** (graded by the Design instructor for the Games category):

- M1U1 ★ UI Standards & Wireframe Baseline — readability standards for the Running Dashboard, the live ghost/ahead-behind feedback, the Challenge Hub and the pet/profile, plus a wireframe baseline; **glanceability while running** is a priority.

- **UI/UX M2:**

- M2U1 ★ Functional UI & Usability — dashboard, personal racing, challenge hub, friends/leaderboards and pet/profile navigable unaided on the Native-Android target.
- M2U2 ★ UX Validation & Final Polished UI — validated, polished non-interruptive-run + pet UX tested with real runners on the Pixel 9a.

- **TEAM M1:**

- M1TA1 ★ Overall Audio Experience — initial audio from the week-5 needs list present & working (run BGM/coaching cues, race-event stings — ghost overtaken / PB split, pet + reward cues), suitable and correctly triggered.
- M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable playback (suited to a screen-off run).

- **TEAM M2:**

- M2TA1 ★ Overall Audio Experience — the app's audio set (run BGM + race-event / milestone / pet / reward / notification SFX, ≥6 types), suited to a non-interruptive, often screen-off run.
- M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the playback the build needs (multi-channel; distinct ghost-overtaken + reward cues).

- M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the content-authoring tool + the GPS-simulation testing tool), prototype demo, team post-mortem (wk14)
-

*Generated by the Project Proposal Assessor skill · grounding: live folder
(Project 2 Technical adjustments/) · references verified 2026-09-22.*